

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
КРЕМЕНЧУЦЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ
ІМЕНІ МИХАЙЛА ОСТРОГРАДСЬКОГО
НАВЧАЛЬНО НАУКОВИЙ ІНСТИТУТ ЕЛЕКТРИЧНОЇ ІНЖЕНЕРІЇ
ТА ІНФОРМАЦІЙНИХ ТЕХНОЛОГІЙ

Кафедра автоматизації та інформаційних технологій

Освітній компонент
АЛГОРИТМИ І СТРУКТУРИ ДАНИХ

ЗВІТ
З ПРАКТИЧНОЇ РОБОТИ № 5

Виконав:

студенти групи КН-25-1

Кремляков Г. І

Перевірив:

старший викладач кафедри АІС

Сидоренко В. М

Тема: Графи. Ациклічні графи

Мета: набути практичних навичок розв'язання задач топологічного сортування орієнтованих ациклічних графів за допомогою алгоритму Кана та оцінювання асимптотичної складності алгоритмів.

Порядок виконання роботи

Умова завдання

Задано ациклічний граф:

$\{1,2,3,4,5,6,7,8,9\}$

$\{(1,2),(1,3),(2,4),(3,5),(4,5),(4,6),(6,7),(7,8),(8,9)\}$

Необхідно побудувати граф і виконати топологічне сортування за допомогою алгоритму Кана.

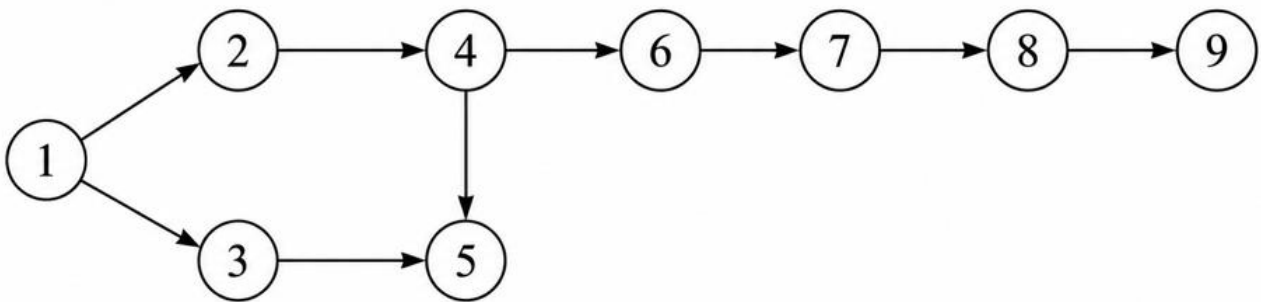


Рисунок 1 – Граф

Розв'язання задачі алгоритмом Кана

Крок 1

Знайдемо вершини без вхідних ребер.

Початково це вершина:

$S=\{1\}$

Список результату:

$L=[]$

Крок 2

Беремо вершину 1.

Додаємо її до результату:

$L=[1]$

Видаляємо ребра:

$(1,2),(1,3)$

Тепер вершини 2 і 3 не мають вхідних ребер:

$S=\{2,3\}$

Крок 3

Беремо вершину 2.

$L=[1,2]$

Видаляємо ребро:

$(2,4)$

Вершина 4 тепер не має вхідних ребер:

$S=\{3,4\}$

Крок 4

Беремо вершину 3.

$L=[1,2,3]$

Видаляємо ребро:

$(3,5)$

Але вершина 5 ще має ребро $(4,5)$, тому не додається.

$S=\{4\}$

Крок 5

Беремо вершину 4.

$L=[1,2,3,4]$

Видаляємо ребра:

$(4,5),(4,6)$

Тепер вершини 5 і 6 не мають вхідних ребер:

$S=\{5,6\}$

Крок 6

Беремо вершину 5.

$L=[1,2,3,4,5]$

Вершина 5 не має вихідних ребер.

$S=\{6\}$

Крок 7

Беремо вершину 6.

$L=[1,2,3,4,5,6]$

Видаляємо ребро:

$(6,7)$

Тепер вершина 7 не має вхідних ребер.

$S=\{7\}$

Крок 8

Беремо вершину 7.

$L=[1,2,3,4,5,6,7]$

Видаляємо ребро:

$(7,8)$

Тепер вершина 8 не має вхідних ребер.

$S=\{8\}$

Крок 9

Беремо вершину 8.

$L=[1,2,3,4,5,6,7,8]$

Видаляємо ребро:

$(8,9)$

Тепер вершина 9 не має вхідних ребер.

$S=\{9\}$

Крок 10

Беремо вершину 9.

$L=[1,2,3,4,5,6,7,8,9]$

Множина S порожня. Алгоритм завершено.

Результат топологічного сортування

$[1,2,3,4,5,6,7,8,9]$

Висновок: у ході виконання практичної роботи було розглянуто орієнтовані ациклічні графи та методи їх топологічного сортування. Було детально вивчено алгоритм Кана та виконано топологічне сортування заданого графа. У результаті отримано правильний порядок обходу вершин, який задовольняє всі залежності між ними. Також було розглянуто алгоритм DFS та проведено порівняння його характеристик з алгоритмом Кана. Отримані знання дозволяють застосовувати топологічне сортування для задач планування, моделювання залежностей та обробки даних.

Відповіді на контрольні запитання

1. Які переваги і недоліки алгоритму Кана порівняно з алгоритмом DFS для топологічного сортування графа?

Алгоритм Кана є простим для розуміння та дозволяє легко визначати наявність циклів у графі. Він добре працює для густо зв'язаних графів. Недоліком є необхідність додатково зберігати кількість вхідних ребер для кожної вершини.

Алгоритм DFS використовує пошук у глибину та часто потребує менше додаткової пам'яті. Він ефективний для розріджених графів, але складніший у реалізації та може викликати переповнення стеку при великій глибині рекурсії.

2. Яка складність часу і пам'яті для кожного з алгоритмів у найгіршому і найкращому випадках?

Для алгоритму Кана:

- Часова складність: $O(|V|+|E|)$
- Складність пам'яті: $O(|V|)$

Для алгоритму DFS:

- Часова складність: $O(|V|+|E|)$
- Складність пам'яті: $O(|V|)$

У найкращому та найгіршому випадках складність залишається однаковою, оскільки необхідно переглянути всі вершини та ребра.

3. Чи можна застосовувати алгоритм Кана до графів з вагами на ребрах? Як це порівняти з DFS?

Так, алгоритм Кана можна застосовувати до графів з вагами, оскільки ваги ребер не впливають на порядок топологічного сортування. Аналогічно і DFS не залежить від ваг ребер. Обидва алгоритми враховують лише напрямки ребер.

4. Як впливає структура графа на швидкість роботи кожного з цих алгоритмів?

У густих графах алгоритм Кана часто працює ефективніше через зручну обробку великої кількості залежностей. DFS краще підходить для розріджених графів із меншою кількістю ребер.

Також глибокі графи можуть уповільнювати DFS через велику глибину рекурсії.

5. Чи є обмеження використання кожного алгоритму для певних типів графів або завдань?

Обидва алгоритми працюють лише для ациклічних орієнтованих графів. Якщо граф містить цикл, топологічне сортування неможливе.

DFS може бути проблемним для дуже великих графів через обмеження рекурсії. Алгоритм Кана потребує додаткової структури даних для зберігання вхідних степенів вершин.

6. Які варіанти оптимізації можна застосувати для кожного алгоритму з метою поліпшення його продуктивності?

Для алгоритму Кана:

- використовувати чергу замість множини;
- ефективно зберігати список суміжності;
- мінімізувати кількість перевірок вхідних степенів.

Для DFS:

- використовувати ітеративну реалізацію замість рекурсії;
- оптимізувати структури даних для обходу графа;
- зменшувати кількість повторних перевірок вершин.